COMPILER CONSTRUCTION LAB · PA1 · GROUP 5 · INTERNAL

Team Prep Pack

Everything the three of us need to present PA1 and survive 10 minutes of questions. The rule from the guidelines: *every member must understand every part of the code.*

Submission deadline: Wed 04.11.2026, 23:59 · Presentation: Thu 12.11.2026 (backup date 19.11.2026) · 15 min talk + 10 min Q&A

1. The 60-second version (everyone memorises this)

"We built the ChocoPy front-end: a JFlex lexer and a CUP LALR(1) parser that produce the JSON AST. The tests compare our tree with the reference parser's tree **down to every line and column**, so most of our work was reverse-engineering the reference's location rules. We did that by using the reference as an **oracle**: we probed it with tiny inputs, and then we built a **random program generator and a differential tester** that applies the grader's own rule. We pass all 33 samples and our 44 edge cases, and more than 3,000 random programs give identical results. The three hardest problems were emitting several DEDENTs from one line (solved with a `yypushback` loop), source locations (node position vs grammar-symbol position), and error recovery. For recovery we went beyond the reference: anchored recovery rules plus Bison-style default reductions, measured with a metric we wrote. We also found three bugs in the reference."

2. The pipeline

Source text `.py` file, read as one String by the driver

› **Lexer** `ChocoPyLexer` (generated from `ChocoPy.jflex`). States YYINITIAL / CODE. Indentation stack.

› **Tokens** `Symbol` s with kind, value, left/right location (`ComplexSymbolFactory`)

› **Parser** `ChocoPyParser` (generated from `ChocoPy.cup`). LALR(1) tables + our actions

› **AST** `chocopy.common.astnodes.*` ; `Program` holds `Errors`

› **JSON** Jackson serialises the tree (driver code, not ours)

Entry point: `chocopy.ChocoPy --pass=s` → `chocopy.pa1.StudentParser.process()` (starter, unchanged) → lexer + parser → `parseProgram()` . With `--pass=r` , the same driver runs the reference instead.

3. Code map: where everything lives

File / lines	What it does	Owner
`ChocoPy.jflex` top comment	The design in one paragraph: two states, the yypushback trick, strings	Mithulram
`ChocoPy.jflex` `%init`	Push the initial 0 onto the indentation stack	Mithulram
`ChocoPy.jflex` `symbol(…)` helpers	Build tokens with locations; record each token's text for error messages	Mithulram
`ChocoPy.jflex` `eofSymbol`	Tokens made up at EOF (one column further right each, like the reference)	Mithulram
`ChocoPy.jflex` `indentationWidth`, `processIndentation`	Tabs to multiples of 8; compare with the stack; INDENT / DEDENT / UNRECOGNIZED	Mithulram
`ChocoPy.jflex` `integerSymbol`, `stringSymbol`	Overflow → UNRECOGNIZED; decode escapes; IDSTRING vs STRING	Mithulram
`ChocoPy.jflex` `<YYINITIAL>` rules	Blank/comment lines ignored; real line → indentation	Mithulram
`ChocoPy.jflex` `<CODE>` rules	Keywords, RESERVED words, identifiers, literals, operators	Mithulram
`ChocoPy.jflex` `<<EOF>>`	Missing NEWLINE, then one DEDENT per open block, then EOF	Mithulram
`ChocoPy.cup` parser code	`syntax_error` (uses token text), `enableDefaultReductions`, `init with`	Dinodi
`ChocoPy.cup` terminals / precedence	Token names = reference names; the precedence table from manual §4.1	Nafeel
`ChocoPy.cup` program, top_decls	Declarations before statements; program location; decl-level `error`	Nafeel
`ChocoPy.cup` class_def, func_def, var_def	Declarations, "<None>" return type, anchored recovery rules	Nafeel / Dinodi
`ChocoPy.cup` stmt, block, elif_else	Statements, elif desugaring, block-end locations	Nafeel
`ChocoPy.cup` expr, cexpr, literal	Expressions: the expr/cexpr split, %prec UMINUS	Nafeel
`tools/difftest.py`	Compare with the reference using the grader's rule (parallel)	Dinodi
`tools/fuzz.py`	Random valid programs + one-defect mutants	Dinodi
`tools/recovery_quality.py`	Metric: errors per real mistake, code kept, aborted parses	Dinodi
`tools/ast_tree.py`	Pretty-print a JSON AST as a tree (for the demo)	Dinodi
`src/test/data/pa1/edge_cases/`	44 hand-written cases + expected `.ast` from the reference	Dinodi
`student_contributed/good.py, bad.py`	Graded: feature coverage; one error per mistake	Dinodi

Owner = who presents it and answers first. Everyone must still be able to explain all of it.

4. Concepts each of us must be able to explain in 30 seconds

Lexical states (YYINITIAL, CODE)

JFlex rules can be limited to a state, and `yybegin(S)` switches state. Our YYINITIAL means "start of a line, indentation not handled yet". CODE means "inside the line".

Longest match, then rule order

JFlex takes the longest possible match. On a tie, the rule written first wins. So `classic` is an identifier (longer than `class`), but `class` alone is the keyword (same length, keyword rule first).

`yypushback(n)`

Gives the last n matched characters back to the input. We use it twice: to un-read the first character of a line, and to re-scan the whitespace once per DEDENT.

Why indentation needs Java code

Matching INDENT/DEDENT needs a stack (counting), and regular expressions cannot count. So the regex only finds the whitespace, and the stack logic is Java in the action.

LALR(1)

A bottom-up parser with one token of lookahead. It *shifts* tokens onto a stack and *reduces* the right-hand side of a rule to its left-hand side. LALR merges LR(1) states with the same core: the tables are smaller, but in rare cases this creates "mysterious" reduce/reduce conflicts (we had none).

Shift/reduce and reduce/reduce conflicts

S/R: in one state the parser could either shift the next token or reduce. Example: `a + b · + c`. Precedence settles it.
R/R: two different reductions are possible. That's a grammar design error (ours came from two error rules, C17).

precedence, nonassoc, %prec

CUP compares the precedence of the rule (its last terminal) with the lookahead token. Higher token → shift, higher rule → reduce. On a tie, `left` → reduce, `right` → shift, `nonassoc` → error (that's how `a<b<c` fails). `%prec UMINUS` gives unary minus the precedence of a dummy token.

xleft / xright

For every labelled symbol `e`, CUP provides `exleft` / `exright`: the location of its leftmost and rightmost *token*. A node spans from the leftmost symbol's xleft to the rightmost symbol's xright.

Node location vs symbol location

The `func_def` symbol includes its closing DEDENT. The `FuncDef` node ends at its body. A class ends at its last member's *symbol*, so it ends at that DEDENT (coverage: method [34,22], class [36,0]).

CUP error recovery

(1) Report the error. (2) Pop states until one can shift `error`. (3) Shift it. (4) Throw away input tokens until the next 3 (`error_sync_size`) parse cleanly. (5) Continue. If it reaches EOF without managing that, it gives up.

Default reductions

In a state whose only possible action is one reduction, do it on *any* lookahead (Bison does this by default). It lets the parser finish a statement before it notices that the next token is wrong. The error position is unchanged.

Desugaring

Turning syntax that is only a convenience into a simpler core form. `elif` → nested `IfStmt` in `elseBody`.

5. Numbers to know by heart

Fact	Value
Provided sample tests / our edge-case tests	33/33 · 44/44
Random programs compared with the reference (valid + mutated)	3,511 files, all identical
Differences the fuzzer found *after* we passed all samples	13 (in the first 600 programs)
LALR conflicts	0
Recovery: exactly one error per real mistake	reference 45% · ours 75%
Recovery: average errors per real mistake	reference 2.0 · ours 1.4
Recovery: parses that gave up (346 programs)	reference 3 · ours 0
Tab width / largest integer	8 · 2,147,483,647
Token names you'll be asked about	EQ (=) vs EQEQ (==), LINDEX/RINDEX ([ ]), IDIV (//), RESERVED, UNRECOGNIZED, IDSTRING
Lines of code	lexer ≈ 450 · grammar ≈ 730 (about half of each is comments)

6. Who presents what (15 minutes)

The deck follows the chair's presentation format (the Metropolis look of the lab's own slides: dark header, orange progress bar) and a thesis-defence structure: an outline slide opens every section.

#	Slide	Speaker	Time
1	Title	Mithulram	0:15
2	Outline	Mithulram	0:15
3	Motivation: matching the reference exactly + scoreboard	Mithulram	0:45
4	Approach: the reference as an oracle (read → probe → build → fuzz → measure)	Mithulram	0:45
5	Outline → Lexer	Mithulram	0:05
6	Challenge 1: one line, many DEDENTs (yypushback loop)	Mithulram	1:10
7	Challenge 2: where is a DEDENT? (locations + EOF quirk)	Mithulram	1:00
8	Challenge 3: strings (build error → redesign → quirks)	Mithulram	0:50
9	Outline → Parser	Nafeel	0:05
10	Grammar: expr/cexpr split, precedence, 0 conflicts	Nafeel	0:50
11	Challenge 4: locations from symbols, not nodes	Nafeel	1:05
12	Challenge 5: elif desugaring	Nafeel	0:35
13	Outline → Error recovery	Dinodi	0:05
14	Challenge 6: cascades → anchored rules	Dinodi	1:00
15	Challenge 7: the vanishing statement → default reductions	Dinodi	1:00
16	Challenge 8: the abort trap	Dinodi	0:55
17	Outline → Evaluation	Dinodi	0:05
18	Evaluation I: differential testing	Dinodi	0:45
19	Evaluation II: recovery quality (chart)	Dinodi	0:40
20	Contribution: three reference bugs	Dinodi	0:35
21	Outline → Conclusion and future work	Nafeel	0:05
22	Limitations and future work	Nafeel	0:40
23	Conclusion	Nafeel	0:25
24	Thank you — questions?	all	
25–26	Backup: CUP recovery algorithm · running it	on demand	

Total ≈ 13:40: Mithulram 4:55 · Nafeel 3:40 · Dinodi 5:05, which leaves over a minute of slack for the hand-overs. Hand-overs: Mithulram → Nafeel after slide 8 ("…so the tokens are right. Nafeel shows what the parser does with them."). Nafeel → Dinodi after slide 12 ("…that's valid programs. Dinodi: what happens when the input is wrong?"). Dinodi → Nafeel for slide 21. The speaker notes in the .pptx contain the full talk track for every slide.

GUIDELINE REMINDERS

- Do **not** explain what ChocoPy, JFlex, CUP or LALR *are*. The audience built the same project. Talk only about *our* problems and fixes.
- Mention what we **could not** fully solve (slide 22). Technical depth is graded.
- Everyone speaks, and everyone answers questions. Never say "that was not my part".
- Attendance is mandatory. If someone is ill, get an official certificate.

7. Q&A protocol

- **First answerer = owner** of that area (code map), but anyone may add: "If I may add one detail…". Keep answers to 20–40 seconds: the answer first, then the reason, then the place in the code.
- **Show the code** when a question is about a specific mechanism. Keep `ChocoPy.jflex` and `ChocoPy.cup` open in the editor, with font size ≥ 18 pt.
- **If you don't know:** "We didn't test that exact case. Based on how the lexer works, I'd expect … We can check it right now." Then run it. A live check is better than a guess.
- **Drill:** each of us answers 10 random questions from the Q&A document *outside* our own area, at least twice before 12.11.

8. Study and rehearsal plan

When	What
Now → Sun 01.11	Each member reads both source files completely (the comments explain every decision) and Documents 1 and 2. Run every command in the Demo Runbook on your own laptop.
Mon 02.11	Team call: everyone explains the other two members' areas (lexer / parser / recovery) in 3 minutes each. Fix anything that's unclear.
Tue 03.11	Final check, then the **submission** (checklist below). One member uploads; the others check the upload.
Wed 04.11, 23:59	**Deadline.** Upload by Tuesday, not on the last evening.
05.11 → 10.11	Two full timed rehearsals (≤ 14:30) + a Q&A drill. Trim slides if you run over.
Wed 11.11	Dress rehearsal with the demo on the presenting laptop. Pre-build the project. Check the HDMI/USB-C adapter.
Thu 12.11	**Presentation.** Arrive early. Laptop charged, project built, slides + PDF backup on a USB stick.

9. Checklists

Submission (by 04.11, 23:59)

- □ Names in README are spelled correctly; hours estimate and contributions are what we agree on.
- □ `mvn clean package` passes; 33/33 samples pass.
- □ `good.py` parses with no errors, and `bad.py` gives ≥ 2 errors on distinct lines.
- □ README line numbers still point at the right code (they move if you edit the files).
- □ `mvn clean` run, no `target/`, no `.git`, no `.DS_Store`.
- □ File named `pa1_group_5.zip`, uploaded to **SubmissionsDir** on Stud.IP.
- □ Download the upload again, unzip it into an empty folder, build it and test it. (The prepared zip is in `PA1/submission/`.)

Presentation day

- □ Laptop charged + charger; HDMI and USB-C adapters.
- □ `.pptx` and `.pdf` of the slides on the laptop **and** on a USB stick.
- □ Project pre-built (`target/assignment.jar` exists), so no network is needed.
- □ Terminal font ≥ 20 pt, opened in `PA1/`, notifications off, Wi-Fi not required.
- □ Editor open on both source files, font ≥ 18 pt.
- □ Demo dry-run done that morning (`bash demo/demo.sh`).
- □ Everyone knows the 60-second version and the slide order.

10. Glossary

Term	Meaning in our project
Logical vs physical line	Physical = text up to a line break. Logical = a physical line with at least one real token (blank/comment lines are not logical lines).
INDENT / DEDENT	Tokens the lexer creates from changes in leading whitespace; they work like { and } around a block.
UNRECOGNIZED	Dummy token for anything the lexer cannot accept. No grammar rule uses it, so it always causes a syntax error.
RESERVED	Python keywords that ChocoPy forbids as names but never uses (lambda, import, …).
IDSTRING	A string literal whose contents look like an identifier (`"Node"`). Allowed as a type annotation.
Oracle	A trusted implementation used to decide the expected output: here, `chocopy-ref.jar`.
Differential testing	Run two implementations on the same input and compare the results.
Mutant	A valid program with one deliberate defect, used to test error handling.
Anchored recovery rule	An error rule with tokens after `error` (e.g. `error NEWLINE`) that tells CUP where to resume.
Consistent state	An LR state with exactly one possible action, which is a reduction.
Cascade	Extra, misleading errors caused by poor recovery from one real mistake.